Lock free Concurrency :-

```
class Counter {
    int count = 0;
    void increment() {
        count++;
    }
}
```

Box 1 → thread t1 → 1000 times
t2 → 1000 times

2000
↳ but less data ha count value → may thread interfere each other & read same value.

Count is made up of 3 memory :-

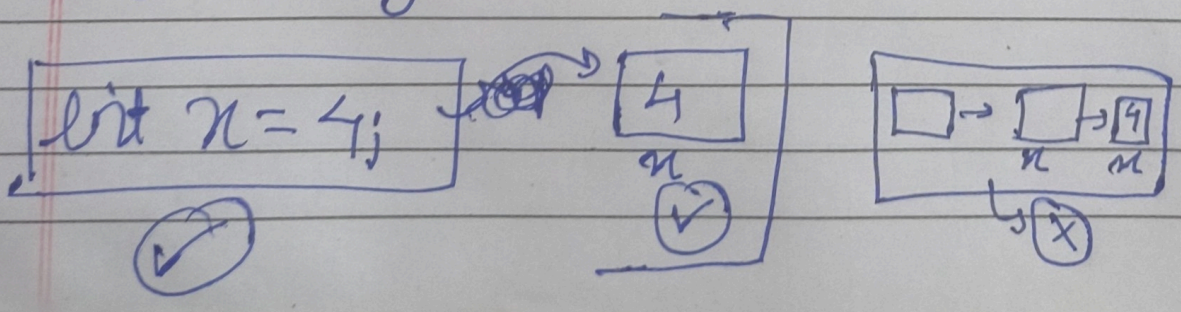
1. Read Memory (temp = count)
2. Increment by 1 (temp = temp + 1)
3. Update Memory (count = temp)

Maybe when count = 0 both t1 and t2 simultaneously read it and increment value by 1 but in seperate it would be 2.

How to Protect?
Box 1 ~~of~~ lock karde ... → By Synchronized Keyword
→ Reentrant lock (Java feature)

Atomic Variables :-

↳ Operation → Single Unit
↳ All or nothing → Puraaam ek baar mahe.



Code:-

```
public class Demo {
public static void main(String[] args) {
Counter counter = new Counter();
Thread t1 = new thread(() -> {
for(int i = 1; i <= 10000; i++) {
counter.increment();
}
});
Thread t2 = new thread(() -> {
for(int i = 1; i <= 10000; i++) {
counter.increment();
}
});
t1.start();
t2.start();
try {
Thread.sleep(2000);
}
catch (InterruptedException e) { }
System.out.println(counter.count);
}}
class counter { int count = 0;
void increment() { count++; }
```

→ t1 & t2 → concurrently working → 2nd thread ka ek saath use karna (same period of time) } Atomic variable they are made in single operation

→ t1 & t2 → parallely ↳ executing at same time.

count.incrementAndGet(), not like count { temp = count; count = temp + 1; count = temp.

→ Without locking mech How?

↳ by using CAS

Methods :-

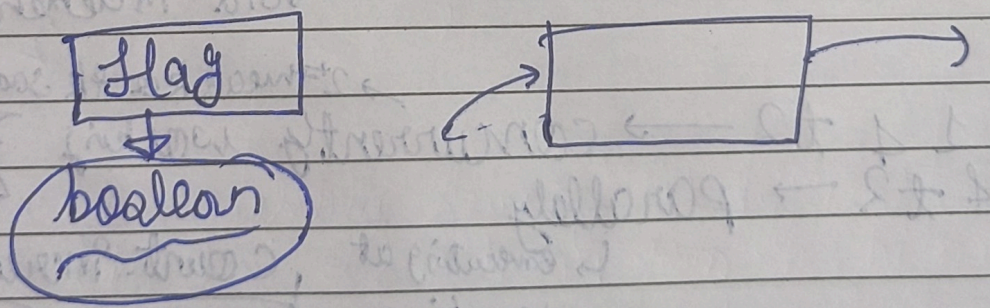
- get()
- set()
- incrementAndGet() → (++x)
- getAndIncrement() (x++)
- decrementAndGet() (--x)
- getAndDecrement() (x--)
- addAndGet(value) (x = x+4)
- getAndAdd(value)

count.get() → to get Integer value

count.set(4)

AtomicLong → same but have higher range.
→ no race condition
good for long range ID generation, unique.

AtomicBoolean :-
→ Producer Customer problem.



Synchronized ? → Overhead (extra cost/work to perform something)
→ Locking Mechanism.

AtomicInteger count = new AtomicInteger(4);

count.incrementAndGet()

↳ yes itself na Atomic integer ha.

If we make 2 atomic together in operation we can't say it's atomic.

```
if (count.get() > 4) {
    count.getAndIncrement();
}
```

↳ Isme bhot chance ha 2 thread same read kar le

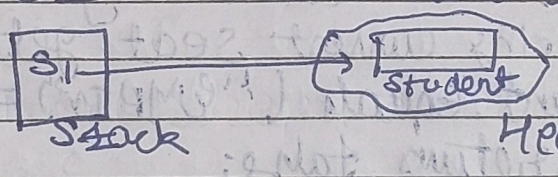
↓

race condition may arise.

Atomic sirf ek statement ko atomic banata hai.

Atomic Reference :-

Student s1 = new Student();



Eg:- Seat Booking :-

```
class SeatBooking {
    String seat = new String("Empty");
    boolean BookSeat (String person) {
        if (seat.equals("Empty")) {
            seat = new String(person);
            return True;
        }
        return false;
    }
}
```

Rahul Rohan dono enter kara saath ma seat empty hai to enter kar a to seat ek ko assign ho and jo thread first run karegi ✓

Methods :-

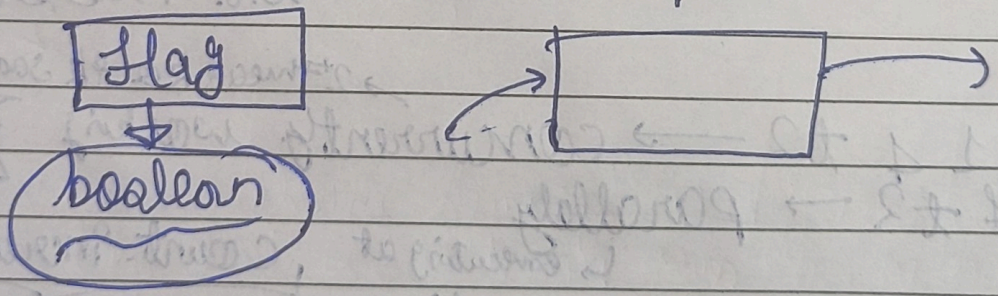
- get()
- set()
- incrementAndGet() → (++x)
- getAndIncrement() (x++)
- decrementAndGet() (--x)
- getAndDecrement() (x--)
- addAndGet(value) (x = x+4)
- getAndAdd(value)

count.get() → to get Intger value

count.set(4)

Atomic long → same but have higher range.
→ no race condition
good for long range ID generation, unique.

Atomic boolean :-
→ Producer Customer problem.



Synchronized ? → Overhead (extra cost/work to perform something)
→ Locking Mechanism

Atomic Integer count = new AtomicInteger(4);

count.incrementAndGet()

↳ yes itself ma atomic integer ha,

If we make 2 atomic to getting in operation] we can't say its atomic.

```
if (count.get() > 4) {
    count.getAndIncrement();
}
```

↳ Some bhot chance ha 2 thread same read kar le

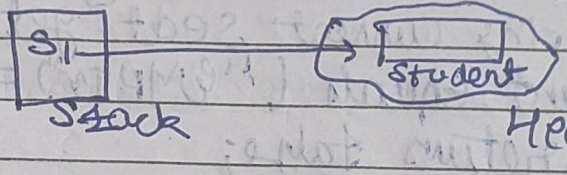
↓

race condition may arise.

Atomic sirf ek statement ko atomic banate hai.

Atomic Refrence :-

```
Student S1 = new Student();
```



Eg:- Seat Booking :-

```
class SeatBooking {
    String seat = new String("empty");
    boolean BookSeat (String person) {
        if (seat.equals("empty")) {
            seat = new String(person);
            return True;
        }
        return false;
    }
}
```

Rahul Rohan] dono enter kara saath ma seat empty hai to enter kar a to seat ek ko assign ho and jo thread first run karegi ✱ O

assume Rahul ko seat degi then phir thread 2 usko override kardegi... par jana ko true return hoga to doria ko false sense of hope hoga.

Multiple Seat Booking Problem.
- Solution 1 :- Locking Mechanism
  - 1 thread locked once.
- Sol 2 :- Lock free Multiconcurrency.

Compare And Set (CAS) operation :-

Atomic Reference :-

```
class SeatBooking {
    AtomicReference<String> seat =
        new AtomicReference<>("EMPTY");
    boolean bookSeat(String name){
        String current seat.get();
    if (current.equals("EMPTY") == false){
        return false;
    }
    return seat.compareAndSet("Empty", name);
    }
}
```

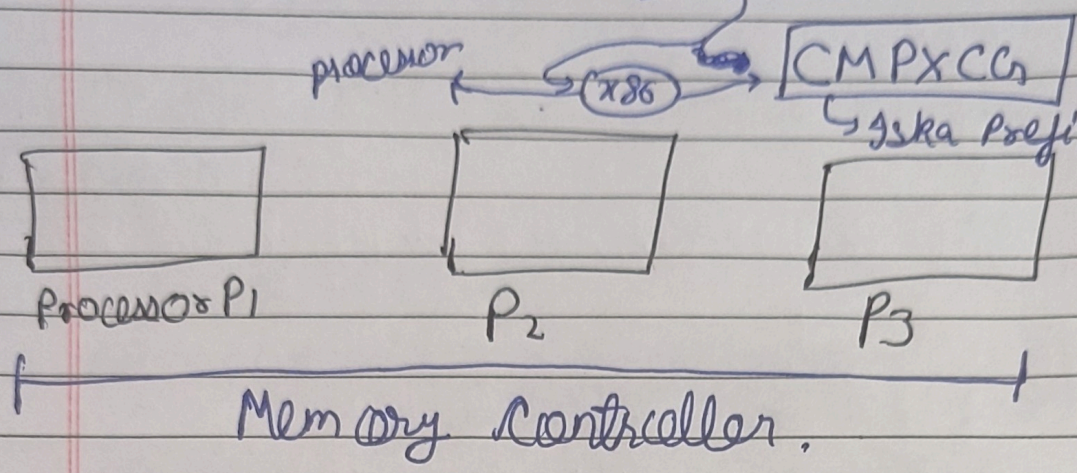
If 4+2 thread parallelly running in different core but this ensure ki kai 2 saath me run nahi sakete, thoda minute difference hoga hi hoga (reason 1 page →)

⭐ Reason :- Parallely work ma thread ek saath execute ho hi sakte :-

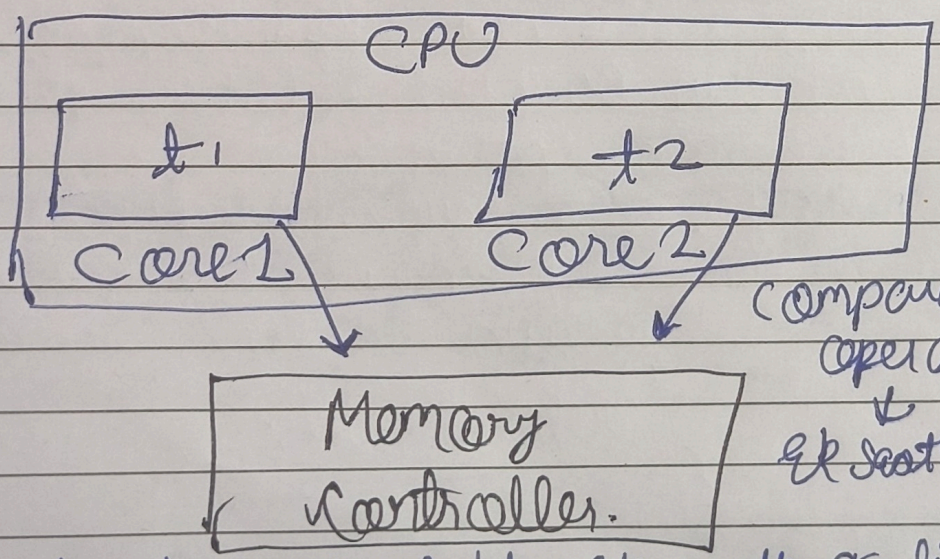
① lock signal :- ② Memory controller :-

↳ Compare And Set (+,-) 1;

↳ CPU gives special instructions to Processor.



When this command (Lock CMPXCG) gets to any processor he tells our system Bus and memory controller that we are performing atomic operation and tells other CPU to stay 1 CPU cycle backwards (pichai).



Ab maine lock signal bhi ek saath aa gya memory Controller ma ek saath but apna Memory Controller ek electronic device ha to usme ek baar ma ek hi electronic signal pass hoga bhale processor ka diff ho.

Electronic device → User → Flip flop → 1 electronic Signal at a time

Parallel Execution [Almost parallel ho but ek CPU ke ka Pork bhi Mem. Controller dekh lega]

Java don't handle Parallel execution Internally.
↳ CPU guarantees that it will prevent.

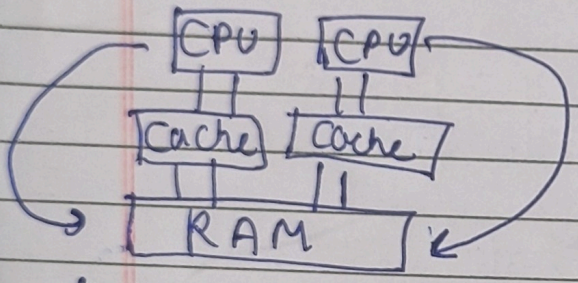
Next Lec:-

diff^n b/w volatile and atomic.

volatile

ATOMIC
↳ either fully executed or not executed

Atomic variable are by default volatile.



[When we want CPU to read from RAM & other than cache we use volatile

Feat. Compare And Set (Expected value, new value) → boolean output

True ← | → False

Electronic device → User → Flip flop → Ek electronic signal at a time

Parallel Execution (Almost parallel ha but ek CPU cycle ka Park bhi Mem. Controller dekh lega)

Java doesn't handle Parallel execution Internally.
↳ CPU guarantees that it will prevent.

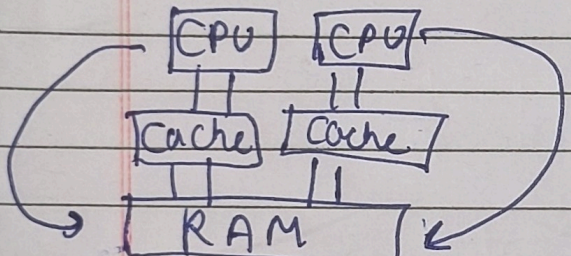
Next Lec:-

diff^n b/w volatile and atomic.

volatile

ATOMIC
↳ either fully executed or not execute

Atomic variable are by default volatile.



(When we want CPU to read from RAM rather than cache we use volatile

Feat. Compare And Set (expected value, new value) → boolean output

True

False